# CAS Box B3 / Rover R04

## ESP32-CAM Edge Computer-Vision Development Report

**Correct project objective:** run computer vision directly on the AI-Thinker ESP32-CAM without a server, laptop, or full OpenCV installation.

The ESP32-CAM should capture a small grayscale image, perform native/optimized edge processing, estimate the path or lane centre, calculate a lateral offset error, and send a compact result to the main ESP32 navigation controller.

# 1. Intended Autonomous Edge Architecture

**ESP32-CAM**
QQVGA/QVGA grayscale
blur/gradient/edges
path centre + error
→
**Main ESP32**
LiDAR, IMU, GPS, ultrasonic
sensor fusion
PID correction
→
**Rover**
bounded motor commands
deadman and safety stops

The phone is for monitoring, operator control, logging, and export. It is not required to perform the vision calculation. The ESP32-CAM is the vision node; the main ESP32 remains the navigation and safety authority.

| Board | Responsibility | Must not do |
| --- | --- | --- |

| | | |
|---|---|---|
| AI-Thinker ESP32-CAM | Capture grayscale frame, run lightweight edge/Canny-style pipeline, estimate lane/path centre, produce compact result. | It must not directly bypass the main ESP32 safety and motor logic. |
| Main ESP32 | Receive vision result, collect LiDAR/IMU/GPS/ultrasonic, fuse data, apply drift correction and motor commands. | Must not trust stale/invalid vision blindly. |
| Phone | Dashboard, logging, image/result display, annotations, ZIP export. | Must not be the only place where control-critical vision runs. |

# 2. Why Resolution and Native Processing Matter

Full OpenCV is unsuitable for the ESP32-CAM memory and CPU budget. Canny-style processing needs image storage for grayscale pixels, blurred pixels, gradients, magnitude, direction, non-maximum suppression, and hysteresis. A 640×480 image contains 307,200 pixels before multiple working buffers are allocated.

| Mode | Pixels | Reason |
|---|---|---|
| QQVGA 160×120 | 19,200 | Safest starting point for several working arrays and fast control updates. |
| QVGA 320×240 | 76,800 | More spatial detail, but higher RAM, CPU, and latency cost. |
| VGA 640×480 | 307,200 | Not appropriate for simultaneous multi-buffer native Canny on this target. |

The correct first implementation is a bounded region of interest, grayscale capture, small fixed-size arrays, integer arithmetic where practical, and a reduced output packet rather than transmitting a

full processed frame for navigation.

# 3. Proposed Native Vision Pipeline

1. Capture a QQVGA or QVGA grayscale frame from the camera.
2. Select a lower image region of interest where the ground/path is expected.
3. Apply a small blur or local smoothing step to suppress camera noise.
4. Calculate horizontal and vertical intensity derivatives.
5. Calculate approximate gradient magnitude and direction.
6. Suppress weak/non-local maxima or use a simplified thresholded edge map.
7. Use hysteresis-style strong/weak edge selection if memory and timing allow.
8. Accumulate edge pixels by column or row.
9. Estimate the path/lane centre from the edge distribution.
10. Calculate `offset_error = detected_centre - image_centre`.
11. Attach confidence, frame sequence, timestamp, and validity state.
12. Send only the compact result to the main ESP32.

**Important design boundary:** the first reliable milestone is a measurable centre/error result. Full image display and advanced vision can remain diagnostic features and must not block the control loop.

# 4. Communication Contract

The camera should send a small binary or compact structured packet, not a large JPEG for every control decision.

| Field | Purpose |
| --- | --- |
| protocol_version | Reject incompatible packets. |
| frame_id | Detect duplicates and gaps. |
| camera_millis | Timestamp the source result. |
| centre_x | Detected path centre in pixels. |

| | |
|---|---|
| image_centre_x | Reference centre. |
| offset_error | Signed lateral image error. |
| edge_count | Amount of usable edge evidence. |
| confidence | Quality of the estimate. |
| valid | Explicit validity flag. |
| processing_time_ms | Measure computational cost. |
| checksum | Detect corrupted packets. |

ESP-NOW is suitable for a compact low-latency result if the boards can be configured with stable peer identity and channel coordination. UDP is simpler when both devices are already on the same Wi-Fi network. The choice must be tested; no communication method should be claimed as complete before packet-loss and latency measurements exist.

# 5. Main-ESP32 Sensor Fusion and Drift Correction

The camera provides a visual path error, not absolute rover pose. The main ESP32 must combine it with its sensor-rich state.

- IMU yaw hold remains the fast heading stabilizer.
- LiDAR provides wall/obstacle geometry and front-stop evidence.
- Ultrasonic sensors provide independent near-field obstacle confirmation.
- GPS supplies outdoor position only when its fix is valid.
- Vision offset provides a lateral path cue when confidence is adequate.
- Wheel velocity commands are adjusted asymmetrically to reduce sideways drift, even without wheel encoders.

Example bounded correction:

```
correction = clamp(K_yaw * yaw_error + K_vision * vision_offset, -
MAX_CORRECTION, +MAX_CORRECTION)
left_command = base_speed - correction
right_command = base_speed + correction
```

Invalid, stale, low-confidence, or missing camera results must be logged and excluded from correction. The system must fall back to the approved sensor/safety behavior rather than fabricate a visual error.

# 6. What We Actually Tried on the Camera

| Work item | Result | Relation to final edge-vision goal |
|---|---|---|
| Camera capture server and wireless snapshot path | Implemented as the camera transport foundation. | Provides the frame source needed before local vision can be validated. |
| Grayscale/low-resolution direction | Identified as the correct memory strategy. | Required for native edge processing. |
| PSRAM-aware buffering and latest-frame acquisition | Added to reduce stale frames and buffer overflow risk. | Important for feeding vision with current frames. |
| Exposure/gain/AEC2 controls and warm-up | Added to improve usable image quality. | Edge detection fails if the image is too dark or unstable. |
| FrameBufferStream compilation fix | Implemented missing `write(uint8_t)`. | Restored the ability to compile the camera transfer code. |
| Asynchronous camera relay | Implemented in source through a background fetch/cache approach. | Prevents image transport from blocking control/heartbeat traffic. |
| Native Canny/path-centre algorithm | Not yet completed or hardware-validated. | This is the major remaining implementation task. |
| Vision result packet to main ESP32 | Not yet completed or measured. | Required before autonomous |

sensor/vision fusion.

# 7. Camera Failures and Lessons

| Failure | Lesson | Correct response |
|---|---|---|
| IMAGE_CAPTURE _REQUESTED followed by IMAGE_MISSING | Displaying/requesting a frame is not the same as receiving and storing it. | Use explicit frame states, binary validation, retries, and manifest reconciliation. |
| FB-OVF messages | Frame acquisition and buffering need bounded ownership and fresh-frame selection. | Use latest-frame capture, warm-up, PSRAM, and release frame buffers reliably. |
| Dark images | Edge detection depends on stable contrast and exposure. | Enable auto exposure/gain and record brightness/exposure metadata. |
| Blocking image transfer | Vision transport must not interfere with real-time rover communication. | Use background capture/cache and compact control packets. |
| Physical enclosure | Camera operation cannot depend on pressing buttons. | All capture, configuration, restart/reconnect, and test actions must be software-controlled. |

# 8. Correct Progress Status

| Goal | Status |
|---|---|
| Run vision without server/laptop | Architecture defined; native vision implementation still required. |

| | |
|---|---|
| Capture small grayscale frames | Camera configuration direction established; needs measured hardware confirmation. |
| Run Canny/edge processing on ESP32-CAM | Not yet delivered; this is the next coding milestone. |
| Estimate lane/path centre | Algorithm specified; not yet validated with real frames. |
| Send compact vision error to main ESP32 | Packet design specified; transport implementation/test pending. |
| Fuse vision with LiDAR/IMU/GPS/ultrasonic | Control concept specified; integration pending. |
| Correct drift without encoders | Requires measured IMU/vision experiments and bounded wheel-command correction. |
| Keep safety authority on main ESP32 | Required architecture; must remain unchanged during implementation. |

**No false completion:** the camera transport work is a foundation, not completion of the edge-vision objective. The actual native edge detector, path-centre estimator, packet link, and fused controller still need to be built and tested.

# 9. Next Implementation Sequence

1. Add a standalone QQVGA grayscale capture test.
2. Add fixed-size grayscale and gradient buffers.
3. Implement thresholded Sobel/Canny-style edge detection.
4. Log processing time, free heap, PSRAM, edge count, and confidence.
5. Draw or transmit a diagnostic centre/error result.
6. Implement and checksum a compact camera-to-main packet.
7. Make the main ESP32 accept vision only when packet age, checksum, and confidence are valid.
8. Fuse vision error with IMU yaw correction using named bounded constants.
9. Run bench tests with the rover lifted, then straight-line drift tests.
10. Only after measured results, consider wall/path or lane behavior.

**Final objective:** an enclosed AI-Thinker ESP32-CAM independently extracts useful visual geometry at the edge, while the main ESP32 combines that result with its sensors and safely controls the rover.